

AI Task Platform — Architecture Document

System Overview

MERN stack platform for asynchronous text transformation tasks.

Components: React frontend, Node.js API, Python worker, MongoDB Atlas, Redis (Upstash) queue, deployed on Kubernetes with ArgoCD GitOps.

Request Flow:

```
User → React Frontend → Node.js Backend → MongoDB Atlas
↓
Redis Queue (Upstash)
↓
Python Worker (x2 replicas)
↓
MongoDB Atlas (update result)
```

Security Measures

Password Hashing (bcrypt): All user passwords are hashed using bcrypt with a salt rounds factor of 10 before storing in MongoDB. Plain-text passwords are never stored or logged.

JWT Authentication: All protected API endpoints require a valid JSON Web Token. Tokens are signed with a secret key, expire after 7 days, and are verified on every request using middleware.

Helmet Middleware: The Express backend uses Helmet.js which sets security-related HTTP headers to protect against common web vulnerabilities like XSS and clickjacking.

Rate Limiting: Authentication endpoints are limited to 20 requests per 15 minutes per IP to prevent brute force attacks. API endpoints are limited to 100 requests per 15 minutes per IP.

No Hardcoded Secrets: All sensitive values (MongoDB URI, Redis password, JWT secret) are stored in environment variables and Kubernetes Secrets. Repository contains only placeholder values.

1. Worker Scaling Strategy

Workers are stateless and use Redis BLPOP — atomic blocking pop ensures two workers never pick the same job. Current setup runs 2 replicas:

```
# Scale workers
kubectl scale deployment/worker --replicas=5 -n ai-task-platform
```

No coordination needed between workers — Redis queue handles job distribution automatically. Each worker independently processes one job at a time.

2. High Task Volume (100k tasks/day)

- 100k/day = ~1.2 tasks/second average, peak ~10/second
 - Redis queue buffers traffic spikes — workers process at steady pace
 - 2 current worker replicas handle normal load, scale to 10 for peak
 - Backend API is stateless — scales horizontally behind Kubernetes service
 - Rate limiting: auth 20 req/15min, API 100 req/15min prevents abuse
 - MongoDB Atlas handles connection pooling automatically
-

3. Database Indexing Strategy

```
// Tasks collection — covers most common query (user tasks, newest first)
taskSchema.index({ user: 1, createdAt: -1 });

// Users collection — fast login lookup + enforces uniqueness
email: { type: String, unique: true }
```

The compound index `taskSchema.index({ user: 1, createdAt: -1 });` covers the most frequent query pattern: fetching all tasks for a specific user sorted by newest first. Without this index MongoDB would do a full collection scan on every request.

4. Handling Redis Failure

Tasks are saved to MongoDB FIRST, then pushed to Redis — guaranteeing zero data loss if Redis is unavailable.

Failure handling in backend:

```
retryStrategy: (times) => {  
  if (times > 3) return null;  
  return Math.min(times * 500, 2000);  
}
```

Failure handling in worker:

- Worker catches Redis `ConnectionError`
 - Falls back to polling MongoDB for `pending` tasks every 3 seconds
 - System continues processing without Redis at reduced performance
 - When Redis recovers, normal queue-based processing resumes automatically
-

5. Staging and Production Environments

Currently deployed to single namespace `ai-task-platform` on Kubernetes.

Production-ready approach:

- Two namespaces: `ai-task-platform-staging` and `ai-task-platform-production`
- Two ArgoCD applications watching different Git branches
- `staging` branch `staging` namespace (for testing)
- `main` branch `production` namespace (live traffic)
- Separate Kubernetes secrets per environment
- Production: higher replica counts (workers: 5, backend: 3)
- CI/CD pipeline deploys to staging first, production after manual approval